

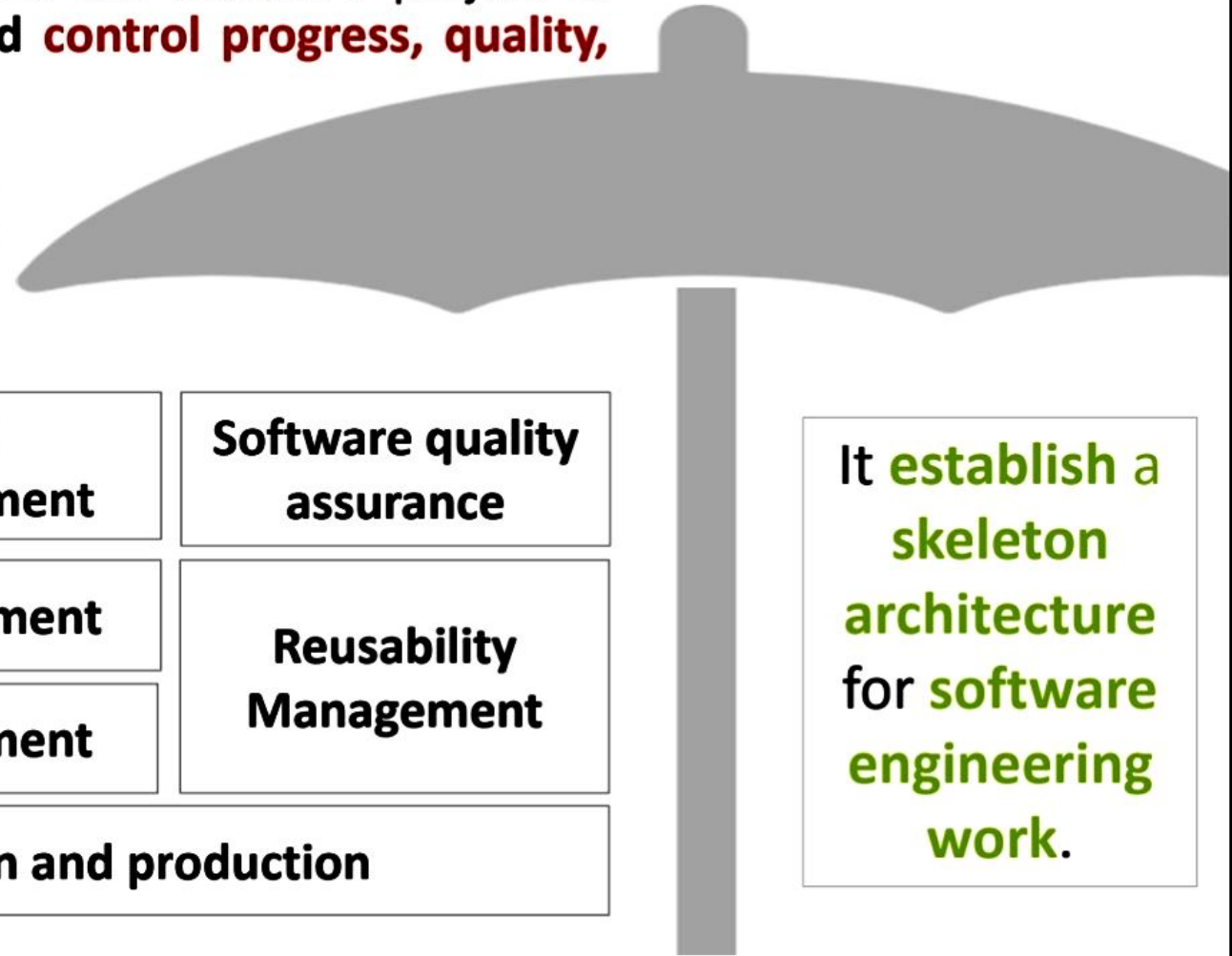
Process Framework Activities (CPMCD)

A process framework establishes the foundation for complete software engineering process, it encompasses five activities

Communication		Communication with Customers / stockholders to understand project requirements for defining software features
Planning		Software Project Plan which defines workflow that is to follow. It describes technical task, risks, resources, product to be produced & work schedule
Modeling		Creating models to understand requirements and shows design of software to achieve requirements
Construction		Code Generation (manual or automated) & Testing (to uncover errors in the code)
Deployment		Deliver Software to Customer Collect feedback from customer based on evaluation Software Support

Umbrella Activities

- ▶ **Umbrella activities** applied throughout the software project & help a software team to manage and **control progress, quality, change & risks**
- ▶ Umbrella activities are those which keep running in the background throughout the software development



**Software project
Tracking & Control**

**Risk
Management**

**Software quality
assurance**

Technical Reviews

Measurement

**Reusability
Management**

Software Configuration Management

Work product preparation and production

It **establish a
skeleton
architecture
for software
engineering
work.**

Umbrella Activities Cont.

- ▶ **Software project tracking and control:** allows the software team to **assess progress against the project plan** and take any necessary action to maintain the schedule.
- ▶ **Risk management:** assesses (**evaluates**) **risks** that may affect the outcome of the project or the quality of the product.
- ▶ **Software quality assurance:** defines and conducts the activities required to **ensure software quality**.
- ▶ **Technical reviews:** **assesses** software engineering **work** products in an effort **to uncover** and remove **errors** before they are propagated to the next activity.
- ▶ **Measurement:** defines and collects process, project and product measures that assist the team in delivering software that meets stakeholders' needs.
- ▶ **Software configuration management:** it manages the effects of change throughout the software process.

Umbrella Activities Cont.

- ▶ **Reusability management:** it defines criteria for work product reuse (including software components) and establishes **mechanisms to achieve reusable components**.
- ▶ **Work product preparation and production:** it encompasses (includes) the activities required to create work products such as **models, documents, logs, forms and lists**.

Software Myths

Beliefs about software and the process used to build it.



“Misleading Attitudes that cause serious problem” are myths.



Management
Myths



Customer Myths



Practitioner's
(Developer)
Myths

Management myth - 1 & 2

We **have standards and procedures** to build a system, which is enough.



We have **the newest computers and development tools.**

Reality

- ▶ Are software **practitioners** aware of standard's existence?
- ▶ Does it **reflect modern software engineering** practice?
- ▶ Is it **complete**?
- ▶ Is it streamlined to **improve time to delivery** while **still maintaining a focus on quality**?
- ▶ In many cases, the answer to all of these questions is "no."

Reality

- ▶ It takes much more than the latest **model** computers to do high-quality software development.
- ▶ **Computer-aided software engineering (CASE)** tools are more important than hardware.

Management myth - 3 & 4

We **can add more programmers** and can catch up the schedule.



I **outsourced the development** activity, now I **can relax** and **can wait** for the **final** working **product**.

Reality

- ▶ Software **development is not a mechanistic process** like manufacturing.
- ▶ In the words of Fred Brooks : "**adding people to a late software project makes it later.**"
- ▶ **People** who were **working** must **spend time educating** the newcomers.
- ▶ People can be added but only **in a planned and well-coordinated** manner.

Reality

- ▶ If an **organization** does **not understand how to manage and control** software projects internally, it will invariably struggle when it outsources software projects.

Customer myth - 1 & 2

A **general statement of objectives** (requirements) is **sufficient** to start a development.



Requirement Changes can be **easily accommodated** because software is very flexible.

Reality

- ▶ Comprehensive (**detailed**) **statements** of requirements is not always possible, an **ambiguous** (unclear) “**statement of objectives**” can lead to disaster.
- ▶ Unambiguous (clear) requirements can be gathered only through effective and continuous communication between customer and developer.

Reality

- ▶ It is true that software **requirements change**, but the **impact** of change **varies with the time** at which it is introduced.
- ▶ When requirements changes are requested early the cost impact is relatively small.

Practitioner's (Developer) myth – 1 & 2

Once we **write** the **program**, our **job** is **done**.



I **can't** access **quality** until it is **running**.

Reality

- ▶ Experts say "the sooner you begin 'writing code', the longer it will take you to get done."
- ▶ Industry data indicates that 60 to 80 % effort expended on software will be after it is delivered to the customer for the first time.

Reality

- ▶ One of the most effective software **quality assurance mechanisms** can be **applied from the beginning** of a project - **the technical review**.
- ▶ Software reviews are more effective "quality filter" than testing for finding software defects.

Practitioner's (Developer) myth – 3 & 4

Working **program** is the **only deliverable** work **product**.

Reality

- ▶ A **working program** is only one **part** of a **software configuration**.
- ▶ A variety of work products (e.g., **models, documents, plans**) provide a foundation for successful engineering and, more important, guidance for software support.



Software engineering is about **unnecessary** documentation.

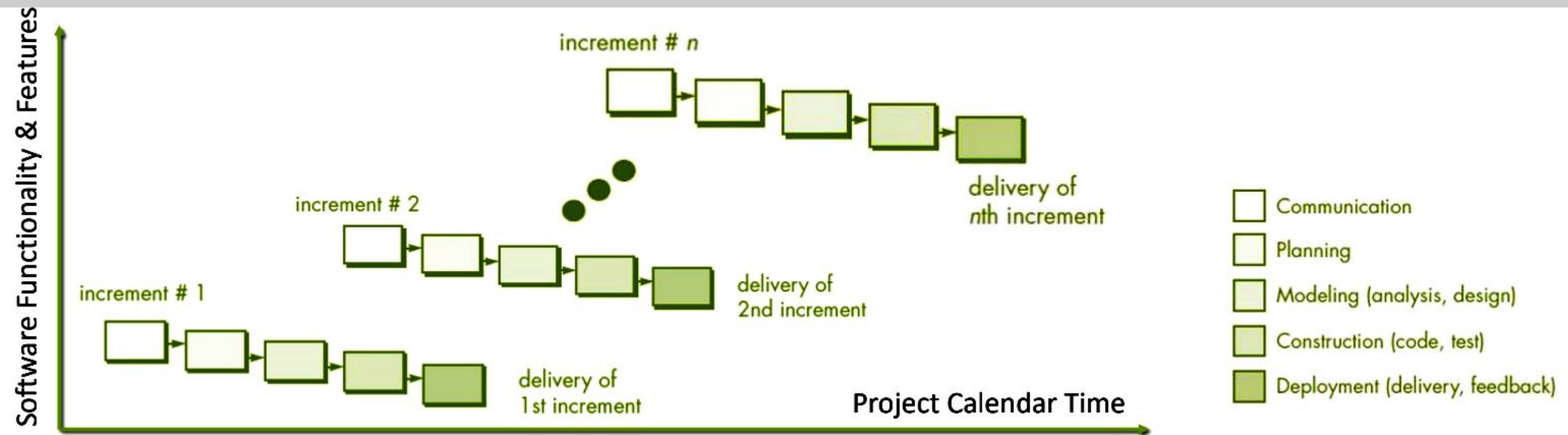
Reality

- ▶ Software engineering is not about creating documents. It is about **creating a quality product**.
- ▶ Better quality leads to reduced rework. And reduced rework results in faster delivery times.

Incremental Process Model

- ▶ There are many situations in which **initial software requirements** are reasonably **well defined**, but the **overall scope of the development** effort prevent a purely linear process.
- ▶ In addition, there may be a **compelling need** to provide a **limited set of software functionality** to users **quickly** and then **refine and expand on that functionality** in later software releases.
- ▶ In such cases, there is a need of a process model that is designed to produce the software in increments.

Incremental Process Model



- ▶ The incremental model **combines** elements of **linear** and **parallel** process flows.
- ▶ This model applies linear sequence in a iterative manner.
- ▶ Initially **core working product** is **delivered**.
- ▶ **Each** linear **sequence** produces deliverable **“increments”** of the software.

Incremental Process Model

e.g. **word-processing software** developed using the **incremental model**

- ▶ It might deliver basic file management, editing and document production functions in the first increment
- ▶ more sophisticated editing in the second increment;
- ▶ spelling and grammar checking in the third increment; and
- ▶ advanced page layout capability in the fourth increment.

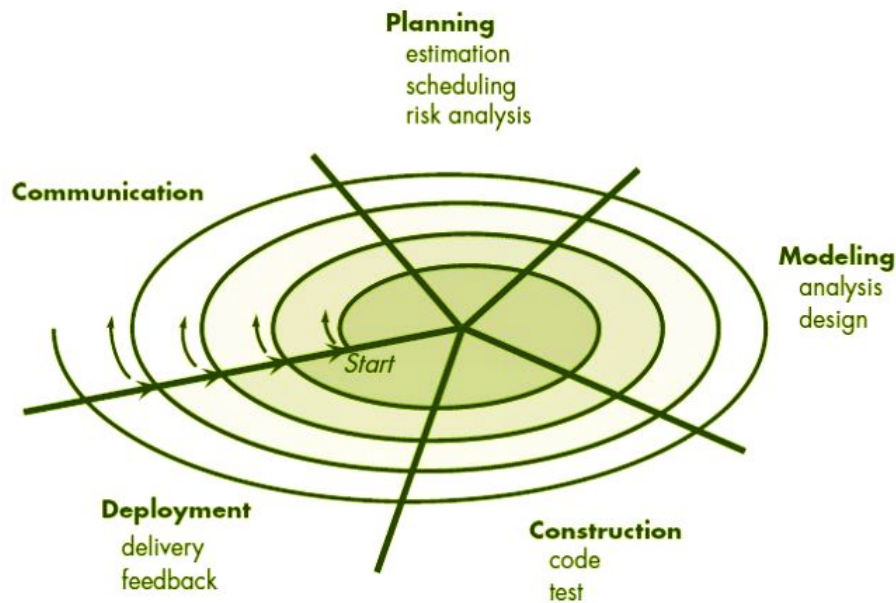
When to use ?

- ▶ When the **requirements** of the **complete** system are clearly **defined** and understood but **staffing is unavailable** for a **complete implementation** by the business deadline.

Advantages

- ▶ Generates **working software quickly** and early during the software life cycle.
- ▶ It is **easier to test** and debug during a smaller iteration.
- ▶ **Customer** can **respond** to each built.
- ▶ **Lowers initial** delivery **cost**.
- ▶ **Easier** to **manage risk** because risky pieces are identified and handled during iteration.

The Spiral Model



- ▶ It provides the **potential** for **rapid development**.
- ▶ Software is developed in a series of evolutionary releases.
- ▶ **Early iteration** release might be **prototype** but **later iterations** provides more **complete version of software**.
- ▶ It is divided into framework activities (C,P,M,C,D). Each activity represent one segment of the spiral
- ▶ **Each pass** through the **planning** region results in **adjustments** to
 - the **project plan**
 - **Cost & schedule** based on feedback

The Spiral Model Cont.

When to use Spiral Model?

- ▶ For development of **large scale / high-risk projects**.
- ▶ When costs and **risk evaluation is important**.
- ▶ Users are **unsure** of their **needs**.
- ▶ **Requirements** are **complex**.
- ▶ New product line.
- ▶ Significant **(considerable) changes** are expected.

Advantages

- ▶ High amount of risk analysis hence, **avoidance of Risk** is enhanced.
- ▶ **Strong approval** and **documentation** control.
- ▶ **Additional functionality** can be **added** at a later date.
- ▶ **Software** is **produced early** in the Software Life Cycle.

Disadvantage

- ▶ Can be **a costly model** to use.
- ▶ Risk analysis **requires highly specific expertise**.
- ▶ Project's success is highly dependent on the risk analysis phase.
- ▶ Doesn't work well for smaller projects.

W⁵HH of Project Management

Boehm suggests **an approach (W⁵HH)** that addresses **project objectives, milestones and schedules, responsibilities, management and technical approaches, and required resources**

Why is the system being developed?

Enables all parties to assess the validity of business reasons for the software work. In another words - does the business purpose justify the expenditure of people, time, and money?

What will be done?

The **answers to these questions** help the team to **establish a project schedule** by **identifying key project tasks** and the **milestones** that are required by the customer

When will it be accomplished?

Project schedule to achieve milestone



W⁵HH of Project Management Cont.

Who is responsible?

Role and responsibility of each member

Where are they organizationally located?

Customer, end user and other stakeholders also have responsibility

How will the job be done technically and managerially?

Management and technical strategy must be defined

How much of each resource is needed?

Develop estimation

It is applicable **regardless of size or complexity** of software **project**

W⁵HH



Function Oriented Metrics

- Function-oriented metrics use a measure of the **functionality delivered** by the application as a normalization value
- Most **widely used metric** of this type is the **Function Point**
 - $FP = \text{Count Total} * [0.65 + 0.01 * \text{Sum (Value Adjustment Factors)}]$
- Function Point **values on past projects** can be **used** to compute,
 - for example, the **average number of lines of code** per function point
- **Advantages**
 - FP is **programming language independent**
 - FP is based on **data** that are **more likely to be known in the early stages** of a project, making it more attractive as an estimation approach

Function Oriented Metrics Cont.

- **Disadvantages**
 - FP **requires** some “sleight of hand” because the **computation** is based on **subjective data**
 - **Counts of the information** domain can be **difficult to collect**
 - FP has **no direct physical meaning**, it’s just a number

Compute Function Points

$$\mathbf{FP} = \text{Count Total} * [0.65 + 0.01 * \sum(F_i)]$$

Count Total is the sum of all FP entries

F_i ($i=1$ to 14) are complexity value adjustment factors (VAF).

Value adjustment factors are used to provide an indication of problem complexity

Information Domain Value	Count		Weighting factor				
			Simple	Average	Complex		
External Inputs (EIs)		×	3	4	6	=	
External Outputs (EOs)		×	4	5	7	=	
External Inquiries (EQs)		×	3	4	6	=	
Internal Logical Files (ILFs)		×	7	10	15	=	
External Interface Files (EIFs)		×	5	7	10	=	
Count total							

COCOMO Model

- **COCOMO (Constructive Cost Estimation Model)** was proposed by Boehm
- According to Boehm, **software cost estimation** should be done through three stages:
 - Basic COCOMO,
 - Intermediate COCOMO, and
 - Complete COCOMO



Basic COCOMO Model

The **basic COCOMO** model gives an **approximate estimate** of the project parameters

The **basic COCOMO estimation** model is given by the **following expressions**

$$\text{Effort} = a_1 + (KLOC)^{a_2} \text{ PM} \quad \text{Tdev} = b_1 \times (\text{Effort})^{b_2} \text{ Months}$$

- **KLOC** is the estimated size of the software product expressed in Kilo Lines of Code,
- **a_1 , a_2 , b_1 , b_2** are constants for each category of software products,
- **Tdev** is the estimated **time to develop** the software, **expressed in months**,
- **Effort** is the **total effort required** to **develop** the software **product**, expressed in **person months (PMs)**.

Project	a_1	a_2	b_1	b_2
Organic	2.4	1.05	2.5	0.38
Semidetached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Basic COCOMO Model Cont.

- The effort estimation is expressed in **units of person-months (PM)**
- It is the **area under the person-month plot** (as shown in fig.)
- An **effort of 100 PM**
 - does not imply that **100 persons** should **work for 1 month**
 - does not imply that **1 person** should be **employed for 100 months**
 - it denotes the **area under the person-month curve** (fig.)

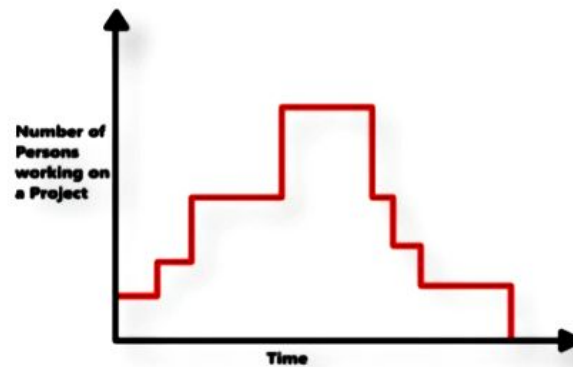


Fig. Person Month Curve

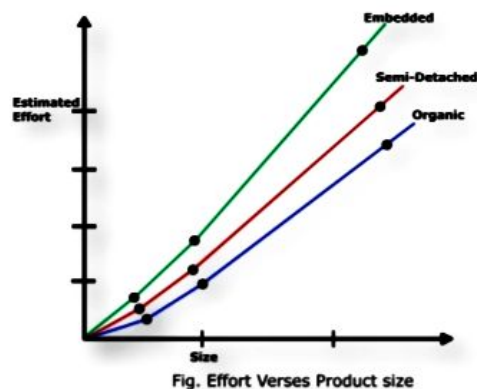
Basic COCOMO Model Cont.

- **Every line of source** text should be **calculated** as **one LOC** **irrespective of** the **actual number of instructions** on that line
- If a **single instruction spans several lines** (say **n lines**), it is **considered** to be **nLOC**
- The values of **a_1 , a_2 , b_1 , b_2** for different categories of products (i.e. organic, semidetached, and embedded) as given by Boehm
- He derived the expressions by examining historical data collected from a large number of actual projects



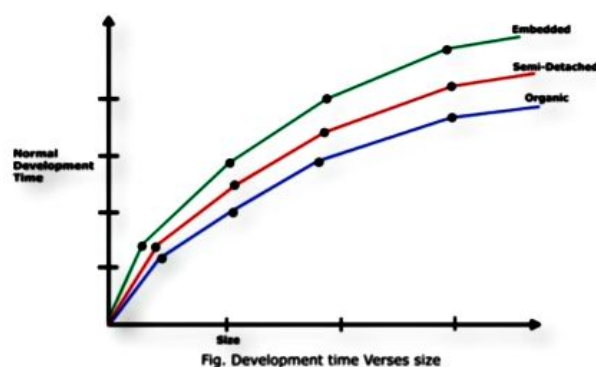
Basic COCOMO Model Cont.

- **Insight** into the **basic COCOMO** model can be obtained by **plotting the estimated characteristics** for different software sizes
- **Fig.** shows a **plot of estimated effort** versus **product size**
- From fig. we can observe that the **effort** is somewhat **superlinear** in the **size of the software** product
- The **effort** required to develop a product **increases** very **rapidly** with **project size**



Basic COCOMO Model Cont.

- The **development time versus the product size** in KLOC is plotted in fig.
- From fig., it can be observed that the **development time** is a **sublinear** function of **the size** of the product
- i.e. when the **size of the product** increases by **two times**, the time to develop the **product does not double** but **risers moderately**
- From fig., it can be observed that the development time is roughly the same for all the three categories of products



Basic COCOMO Model Cont.

- **Effort** and the **duration estimations** obtained **using** the **COCOMO** model are called as **nominal effort** estimate and nominal **duration estimate**
- The term **nominal implies** that
 - if anyone **tries to complete the project** in a **time shorter** than the **estimated** duration, then the **cost will increase drastically**
 - But, if anyone **completes the project over a longer period** of time than the **estimated**, then there is almost **no decrease** in the **estimated cost value**



Basic COCOMO Model Cont.

Example:

Assume that the **size** of an **organic type** software product **has been estimated** to be **32,000 lines of source code**. Assume that the **average salary** of software **engineers** be **Rs. 15,000/- per month**. **Determine** the **effort required** to develop the software product **and** the **nominal development time**

$$\text{Effort} = a_1 + (KLOC)^{a_2} \text{ PM}$$

$$= 2.4 + (32)^{1.05} \text{ PM}$$

$$= 91 \text{ PM}$$

$$\text{Tdev} = b_1 \times (\text{Effort})^{b_2} \text{ Months}$$

$$= 2.5 \times (91)^{0.38} \text{ Months}$$

$$= 14 \text{ Months}$$

Cost required to develop the product = 14 x 15000 = Rs. **2,10,000/-**

Intermediate COCOMO model

- The **basic COCOMO** model **assumes** that **effort** and **development time** **are functions** of the **product size alone**
- However, a **host of other** project **parameters** besides the product size **affect** the **effort required** to develop the product as well as the **development time**
- Therefore, **in order to obtain an accurate estimation** of the effort and project **duration**, the **effect** of all relevant **parameters** must be **taken** into **account**
- The **intermediate COCOMO** model **recognizes this fact** and refines the initial estimate obtained using the basic COCOMO expressions **by using a set of 15 cost drivers (multipliers)** based on **various attributes** of software development
 - For example, if modern programming practices are used, the initial estimates are scaled downward by multiplication with a cost driver having a value less than 1

Intermediate COCOMO model Cont.

- It **requires** the project manager **to rate** these **15** different **parameters** for a particular **project** on a **scale of one to three**.
- Then, **depending** on these **ratings**, **appropriate cost driver values** which should be **multiplied with the initial estimate** obtained using the basic COCOMO.

The cost drivers can be **classified** as being attributes of the following items

Product: The characteristics of the product that are considered include the **inherent complexity** of the product, **reliability requirements** of the product, etc.

Computer: Characteristics of the computer that are considered include the **execution speed** required, **storage space** required etc.

Intermediate COCOMO model Cont.

Personnel: The attributes of development personnel that are considered include the **experience level of personnel**, **programming capability**, **analysis capability**, etc.

Development Environment: Development environment attributes capture the **development facilities available** to the developers. An important parameter that is considered is the **sophistication of the automation (CASE) tools used** for software development



Complete COCOMO model

- A major **shortcoming** of both the **basic** and **intermediate COCOMO** models is that they **consider** a software product **as a single homogeneous entity**
- Most **large systems** are **made** up several **smaller sub-systems**
- These **sub-systems** may have widely **different characteristics**
 - E.g., **some sub-systems** may be considered as **organic type**, some **semidetached**, and some **embedded**
 - Also for **some subsystems** the **reliability requirements** may be **high**, for some the **development team** might have **no previous experience** of similar development etc.
- The **complete COCOMO** model **considers** these differences in **characteristics** of the **subsystems** and **estimates** the effort and development time **as the sum of the estimates for the individual subsystems**



Complete COCOMO model Cont.

- The **cost** of each **subsystem** is **estimated separately**
- This approach **reduces** the **margin of error** in the final **estimate**

Risk

A risk is a **potential (probable) problem** – which might happen and might not



Conceptual definition of risk

- Risk **concerns future happenings**
- Risk **involves change** in mind, opinion, actions, places, etc.
- Risk **involves** choice and the **uncertainty** that choice entails

Two characteristics of risk

Uncertainty

The risk **may or may not happen**, so there are no 100% risks (some of those may called constraints)

Loss

If the risk becomes a reality and **unwanted consequences or losses** occur

Risk Categorization: Approach-1

■ Project risks

- They **threaten** the **project plan**
- If they become real, it is likely that the **project schedule will slip** and that **costs will increase**

■ Technical risks

- They **threaten the quality** and **timeliness** of the software to be produced
- If they become real, **implementation may become difficult** or impossible

■ Business risks

- They **threaten** the **feasibility** of the **software** to be built
- If they become real, they **threaten the project** or the **product**



Risk Categorization: Approach-1

Sub-categories of Business risks

- **Market risk**
 - Building an **excellent product** or system that **no one really wants**
- **Strategic risk**
 - Building a **product that no longer fits into** the overall **business strategy** for the company
- **Sales risk**
 - Building a **product** that the **sales force doesn't understand** how to **sell**
- **Management risk**
 - **Losing** the **support** of **senior management** due to a change in focus or a change in people
- **Budget risk**
 - **Losing budgetary** or **personnel commitment**



Risk Categorization: Approach-2

- **Known risks**
 - Those **risks** that can be **uncovered after careful evaluation** of
 - the **project plan**,
 - the **business and technical environment** in which the project is being developed, and
 - other **reliable information sources** (Ex. unrealistic delivery date)
- **Predictable risks**
 - Those **risks** that are **deduced** (draw conclusion) from **past project** experience (Ex. past turnover)
- **Unpredictable risks**
 - Those **risks** that can and do **occur**, but are **extremely difficult** to **identify** in **advance**



RMMM

- **RMMM** - Mitigation, Monitoring, and Management
- An **effective strategy** for **dealing with risk** must consider three issues
 - Risk **mitigation** (i.e., **avoidance**)
 - Risk **monitoring**
 - Risk **management** and **contingency planning**

RMMM

Risk Mitigation is a **problem avoidance activity**

Risk Monitoring is a **project tracking activity**

Risk Management includes **contingency plans** that risk will occur



RMMM PLAN

- The RMMM **PLAN documents** all work performed as part of **risk analysis** and used by the project manager as part of the **overall project plan**
- Some software teams do not develop a formal RMMM document, rather each risk is documented individually using a **Risk information sheet (RIS)**
- In most cases, RIS is maintained using a database system.
- So Creation and information entry, priority ordering, searches and other analysis may be accomplished easily.
- The format of RIS is describe in diagram

SRS (Software Requirements Specification)

Software Requirement Specification (SRS) is a **document that completely describes what the proposed software should do without describing how software will do it.**

SRS is also helping the clients to understand their own needs.

SRS Contains

A **complete information** description

A **detailed functional** description

A representation of **system behaviour**

An indication of **performance requirements and design constraints**

Appropriate **validation criteria**

Other information suitable to requirements

Characteristics of a Good SRS

- ▶ SRS should be **accurate, complete, efficient, and of high quality**, so that it does not affect the entire project plan.
- ▶ An SRS is **said to be of high quality when the developer and user easily understand the prepared document.**
- ▶ **Characteristics of a Good SRS:**

Correct

SRS is correct when **all user requirements are stated in the requirements document.**

Note that there is **no specified tool or procedure to assure the correctness of SRS.**

Unambiguous

SRS is unambiguous when **every stated requirement has only one interpretation.**

Complete

SRS is complete when the **requirements clearly define what the software is required to do.**

Characteristics of a Good SRS Cont.

Ranked for Importance/Stability

All requirements are not equally important, hence **each requirement is identified to make differences** among other requirements.

Stability implies the probability of changes in the requirement in future.

Modifiable

The requirements of the user can change, hence requirements document should be created in such a manner that those **changes can be modified easily**.

Traceable

SRS is traceable when the **source of each requirement is clear** and facilitates the reference of each requirement in future.

Verifiable

SRS is verifiable when the **specified requirements can be verified with a cost-effective process** to check whether the final software meets those requirements.

Consistent

SRS is consistent when the **subsets of individual requirements defined do not conflict** with each other.

Problems Without SRS

- ▶ Without developing the SRS document, the **system would not be properly implemented** according to customer needs.
- ▶ Software developers would not know whether what they are developing is **what exactly is required by the customer**.
- ▶ Without SRS, it will be very difficult for the **maintenance engineers to understand the functionality** of the system.
- ▶ It will be very difficult for **user document writers to write the users' manuals properly** without understanding the SRS.

Class diagram

The **purpose** of class modeling is to describe **objects in systems** and **different types of relationships between them**.

The class diagram is used to **construct** and **visualize** object-oriented systems.

- ▶ Class modeling is used to **specify** the **structure of the objects, classes, or components** that exist **in the problem domain** or system.
- ▶ Class diagram provides a **graphic notation for modeling classes and their relationships**.
- ▶ Class is a **blueprint of an object**.
- ▶ An object is a **concept, abstraction, or thing with an identity** that has **meaning for an application**.
- ▶ Class diagrams represent an overview of the system like **classes, attributes, operations, and relationships**.

Elements of Class Diagram (Class Name)

Class Name
Attributes
Operations

- ▶ The **name** of the class appears in the **upper section**.
- ▶ Class name should be **meaningful**.
- ▶ Class name should always be aligned **center** of the upper section.
- ▶ Class name should **start with capital letters**, and **intermediate letter is a capital**.
- ▶ Class name should be always **bold format**.
- ▶ For e.g.:

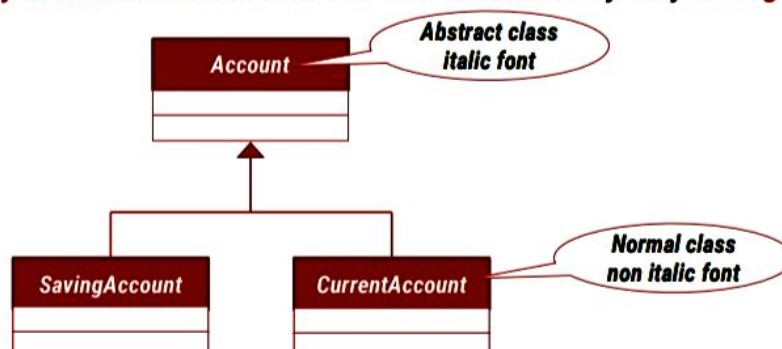
Account

Customer

Employee
- ▶ Abstract class name should be written in **italic format**.

Elements of Class Diagram (Class Name) Cont.

- ▶ For e.g. in the **banking system**, there are **two types of accounts**; one is a **saving account** and another is a **current account**.
- ▶ **Account** is an **abstract class** and **saving account** and the **current account** is a subclass of **Account**.
- ▶ The system **can't directly access** the **Account** class. It is accessible by only **saving accounts** and **current accounts**.



Elements of Class Diagram (Attributes)

Class Name
Attributes
Operations

- ▶ An attribute is a named **property of a class** that describes a value held by each object of the class.
- ▶ The UML notation lists attributes in the **second compartment** of the class box.
- ▶ The attribute name should be in the **regular face, left align** in the box & use the **lowercase letters** for the **first character**.
- ▶ The **data type** for the attribute should be written **after the colon**.
- ▶ **Accessibility** of attribute must be defined using a member access modifier.
- ▶ Syntax : **accessModifier attributeName:dataType=defaultValue**
- ▶ For e.g. in this example '-' represents private access modifier

Account
- accountNumber:long

Customer
- customerName:String

Employee
- employeeName:String

Elements of Class Diagram (Access Modifiers)

- ▶ **Public (+):** Member accessible by **all classes**, whether these classes are in the same package or in another package.
- ▶ **Private (-):** Member **cannot be accessed outside** the enclosing/declaring class.
- ▶ **Protected (#):** Member can be **accessed only by subclasses** and within a class.
- ▶ **Package (~):** Member can be accessible by all classes, **within the package**. Outside package member not accessible.
- ▶ **Static (underlined) :** Member can be **accessed using class name only**.
- ▶ In example you can see how to use access specifier

SavingAccount
+ accountNumber:long
+ name:String
dob: Date
~ panNumber:String

Elements of Class Diagram (Operation)

Class Name
Attributes
Operations

- ▶ The operation is a **function or procedure** that may be applied to objects in a class.
- ▶ The UML notation is to list operations in the **third compartment** of the class box.
- ▶ The operation name in the **regular face, left align** the name in the box, and use a **lowercase letter** for the **first character**.
- ▶ Optional detail, such as an argument list and result type, may follow each operation name.
- ▶ The **return type** of method should be written after colon.
- ▶ **Accessibility** of operation must be defined using a member access modifier.
- ▶ Syntax : **accessModifier methodName(argumentList):returnType**

For e.g.: you can see **change phone number** is a **method** that accepts **phone number** as an **argument** and **return** the **int value** as a response.

Account
+ changePhoneNumber(phoneNumber:String):int

Activity Diagram

An activity diagram visually presents a **series of operation or flow of control** in a system similar to algorithm or a flowchart.

- ▶ An activity diagram is like a **traditional flowchart** in that it show the **flow of control from step to step**.
- ▶ An activity diagram can **show both sequential and concurrent flow of control**.
- ▶ Activity diagram **mainly focus on the sequence** of operation rather than on objects.
- ▶ Activity diagram represent the **dynamic behavior** of the system or part of the system.
- ▶ An activity diagram shows '**How**' system works.
- ▶ Activity diagram are most **useful during early stages of designing algorithms and workflows**.

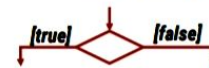
Elements of Activity Diagram

Activity

Activity

- ▶ The **main element** of an activity diagram is the activity itself.
- ▶ An activity is a **function/operation performed by the system**.
- ▶ The **elongated ovals** show activities.
- ▶ An **unlabeled arrow from one activity to another activity**, that indicates that the **first activity must complete before the second activity begin**.

Branches



- ▶ If there is more than one successor to an activity, each arrow may be labeled with a condition in square brackets. For e.g. [failure]
- ▶ As a notational convenience, a **diamond shows a branch** into multiple successors.
- ▶ The diamond has one incoming arrows and two or more outgoing arrows. Each with condition.

Elements of Activity Diagram Cont.

Initiation



- ▶ A **solid circle with an outgoing arrow** shows the **starting point** of an activity diagram.
- ▶ When an activity diagram is activated, control starts at the solid circle and proceeds via the outgoing arrow toward the first activities.

Termination



- ▶ A **bull's eye** – a solid circle surrounded by a hollow circle shows the **termination point**.
- ▶ The symbol **only has incoming arrows**.
- ▶ When control reaches a bull's eye, the overall **activity is complete** and **execution of the activity diagram ends**.

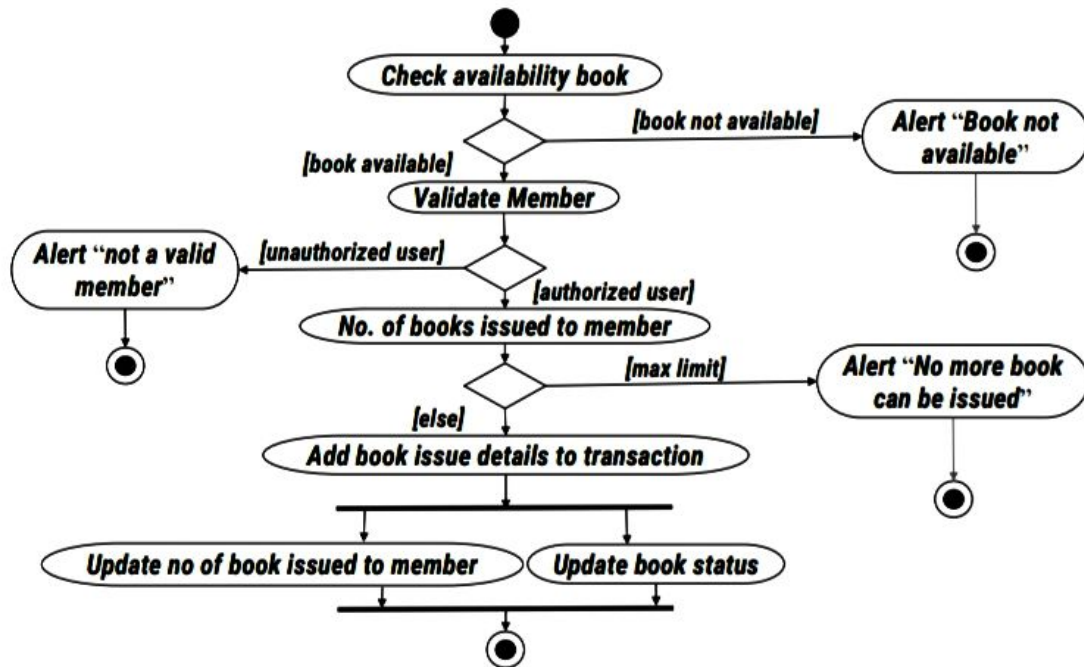
Elements of Activity Diagram Cont.

Concurrent Activities

- ▶ System can perform **more than one activity at a time**.
- ▶ For e.g. one activity may be followed by another activity, then **split into several concurrent activities** (a **fork of control**), and finally be **combined into a single activity** (a **merge of control**).
- ▶ A fork or merge is shown by a **synchronization bar** –a heavy line with one or more input arrows and one or more output arrows.



Activity Diagram for Book Issue



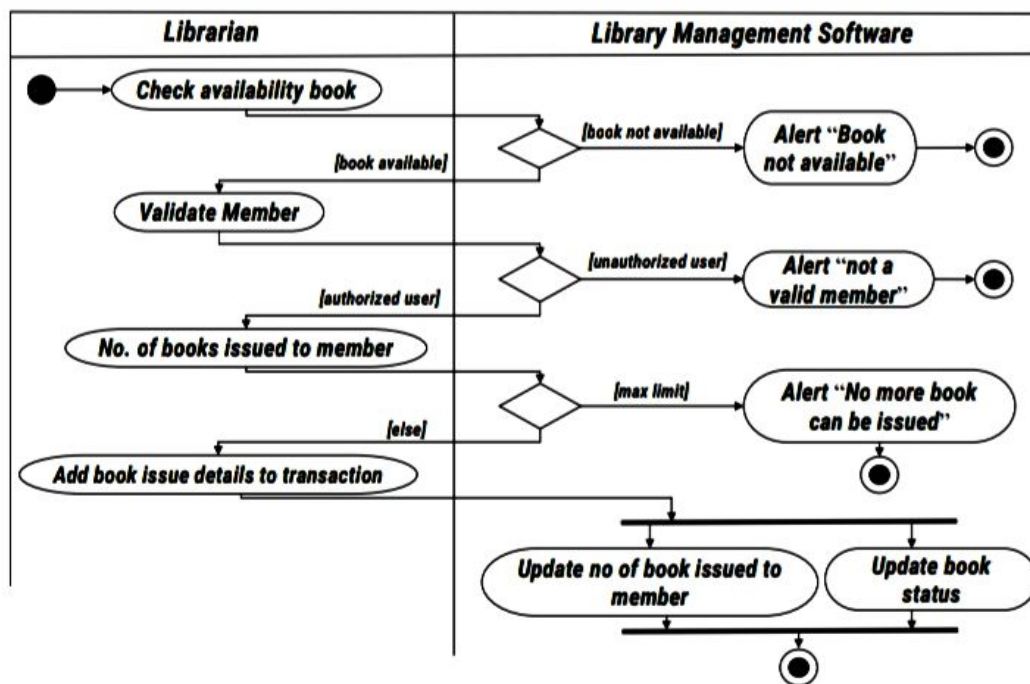
Swimlane Diagram

- ▶ In a business model, it is often useful to know **which human department is responsible for an activity**.
- ▶ When design of the system is complete, the activity will be **assigned to a person/department**, but at a high level it is **sufficient to partition the activities among departments**.
- ▶ You can show such a partitioning with an activity diagram by **dividing in to columns and lines**.
- ▶ Each column is called **swim-lane** by analogy to a swimming pool.
- ▶ Placing an **activity** within a **particular swim-lane** indicates that is **performed by a person/department**.
- ▶ Lines across swim-lane **boundaries** indicate **interaction among different person/department**.

How to Draw a Swimlane Diagram

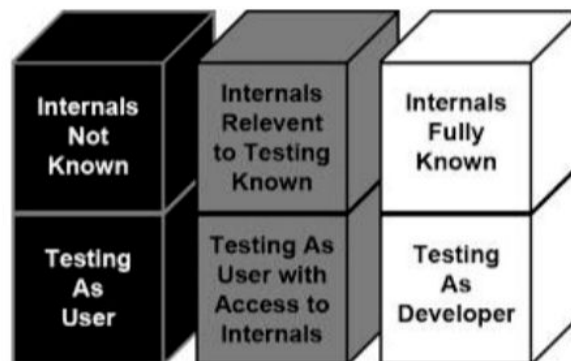
- ▶ **Step 1:** Identify the various activities and actions your business process or system
- ▶ **Step 2:** Figure out which person/departments are responsible for the completion of activity.
- ▶ **Step 3:** Figure out in which order the actions are processed.
- ▶ **Step 4:** Figure out who is responsible for each action and assign them a swimlane and group each action they are responsible for under them

Swimlane Diagram for Book Issue



Views of Test Objects

Black Box Testing	White Box Testing	Grey Box Testing
Close Box Testing Testing based only on specification	Open Box Testing Testing based on actual source code	Partial knowledge of source code



Black Box Testing

- ▶ Also known as **specification-based testing**
- ▶ **Tester** has **access** only to **running code** and the **specification** it is supposed to satisfy
- ▶ **Test cases** are **written with no knowledge of internal workings** of the code
- ▶ **No access** to **source code**
- ▶ So **test cases don't worry** about **structure**
- ▶ **Emphasis** is only on **ensuring** that the **contract is met**

Advantages

- ▶ **Scalable**; not dependent on size of code
- ▶ Testing **needs no knowledge** of **implementation**
- ▶ **Tester** and **developer** can be **truly independent** of each other
- ▶ **Tests** are **done** with **requirements in mind**
- ▶ Does **not excuse inconsistencies** in the **specifications**
- ▶ **Test cases** can be **developed** in **parallel** with **code**

Black Box Testing Cont.

Disadvantages

- ▶ **Examine pre-condition**, and **identify** equivalence classes
- ▶ **All possible inputs** such that all classes **are covered**
- ▶ **Apply the specification to input** to write down **expected output**

Test Case Design

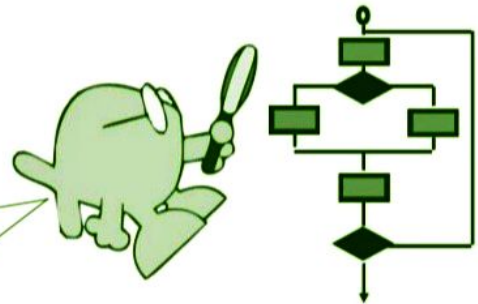
- ▶ **Test size** will **have to be small**
- ▶ **Specifications** must be **clear, concise, and correct**
- ▶ May **leave** many **program paths untested**
- ▶ **Weighting** of program **paths** is **not possible**



White Box Testing

- ▶ Also known as **structural testing**
- ▶ White Box Testing is a software testing **method** in which the **internal structure/design/implementation** of the module being tested **is known to the tester**
- ▶ Focus is on **ensuring** that even **abnormal invocations** are **handled gracefully**
- ▶ Using white-box testing methods, you can **derive test cases** that
 - **Guarantee** that all **independent paths** within a module have been **exercised at least once**
 - **Exercise** all **logical decisions** on their true and false sides
 - **Execute** all **loops** at their boundaries
 - **Exercise internal data structures** to ensure their validity

...our goal is to **ensure** that **all statements and conditions** have been **executed at least once** ...



White Box Testing Cont.

- ▶ It is applicable to the following levels of software testing
 - **Unit Testing:** For testing *paths within a unit*
 - **Integration Testing:** For testing *paths between units*
 - **System Testing:** For testing *paths between subsystems*

Advantages

- ▶ **Testing** can be **commenced** at an **earlier stage** as one need not wait for the GUI to be available.
- ▶ **Testing** is **more thorough**, with the possibility of **covering most paths**

Disadvantages

- ▶ Since **tests** can be very **complex**, **highly skilled resources** are **required**, with thorough **knowledge** of programming and implementation
- ▶ **Test script maintenance** can be a **burden**, if the implementation changes too frequently
- ▶ Since this method of testing is closely tied with the application being testing, **tools to cater to every kind of implementation/platform** may not be readily available

White-box testing strategies

- ▶ One white-box testing strategy is said to be stronger than another strategy, if all types of errors detected by the first testing strategy is also detected by the second testing strategy, and the second testing strategy additionally detects some more types of errors.

White-box testing strategies

1 **Statement** coverage

2 **Branch** coverage

3 **Path** coverage

Statement coverage

- ▶ It aims to design test cases so that **every statement in a program is executed at least once**
- ▶ Principal idea is **unless a statement is executed**, it is very hard to determine if an **error exists** in that statement
- ▶ Unless a statement is executed, it is very difficult to observe whether it causes failure due to some illegal memory access, wrong result computation, etc.

White-box testing strategies Cont.

Consider the Euclid's **GCD computation** algorithm

```
int compute_gcd(x, y)
int x, y;
{
1   while (x != y){
2   if (x > y) then
3       x = x - y;
4   else y = y - x;
5   }
6   return x;
}
```

By choosing the test set **{{(x=3, y=3), (x=4, y=3), (x=3, y=4)}**, we can exercise the program such that all statements are executed at least once.

White-box testing strategies Cont.

Branch coverage

- ▶ In the branch coverage based testing strategy, **test cases are designed to make each branch condition to assume true and false values in turn**
- ▶ It is also known as **edge Testing** as in this testing scheme, each edge of a program's control flow graph is traversed at least once
- ▶ Branch coverage **guarantees statement coverage**, so it is stronger strategy compared to Statement Coverage.

Path Coverage

- ▶ In this strategy test cases are executed in such a way that **every path is executed at least once**
- ▶ All possible control paths taken, including
 - All loop paths taken **zero, once** and **multiple items** in technique
 - The **test case** are **prepared** based on the **logical complexity measure** of the **procedure design**
- ▶ **Flow graph, Cyclomatic Complexity** and **Graph Metrics** are used to arrive at basis path.

White-box testing strategies Cont.

Branch coverage

- ▶ In the branch coverage based testing strategy, **test cases are designed to make each branch condition to assume true and false values in turn**
- ▶ It is also known as **edge Testing** as in this testing scheme, each edge of a program's control flow graph is traversed at least once
- ▶ Branch coverage **guarantees statement coverage**, so it is stronger strategy compared to Statement Coverage.

Path Coverage

- ▶ In this strategy test cases are executed in such a way that **every path is executed at least once**
- ▶ All possible control paths taken, including
 - All loop paths taken **zero, once** and **multiple items** in technique
 - The **test case** are **prepared** based on the **logical complexity measure** of the **procedure design**
- ▶ **Flow graph, Cyclomatic Complexity** and **Graph Metrics** are used to arrive at basis path.

Types of Software Maintenance Cont.

Perfective Maintenance



- This includes **modifications** done in order to **keep** the **software usable over long period of time**
- It **includes** new features, new user requirements for refining the software and **improve its reliability** and **performance**.
- This includes **changing** the **functionalities** of as per the **user's changing needs**

Preventive Maintenance



- **Modifications** to **prevent future problems** of software
- It aims to **attend problems**, which are **not significant at this moment** but may cause serious issues in future
- It comprises **documentation updating**, **code optimization** and **code restructuring**.

Re-Engineering

- When we **need** to **update the software** to keep it to the current market, **without impacting** its **functionality**, it is called **software re-engineering**
- It is a **process** where the **design** of software is **changed** and **programs** are **re-written**
- **Legacy software** cannot keep **tuning** with the latest technology available in the market
 - For example, **initially UNIX** was developed in **assembly language**. **When** language **C came** into existence, **UNIX** was **re-engineered** in **C**, because working in assembly language was difficult.
- Other than this, **sometimes programmers notice** that few **parts** of **software** need **more maintenance** than others and they also **need re-engineering**



Reverse Engineering

- **Reverse engineering** can **extract design information** from **source code**
- The **abstraction level** of a **reverse engineering process** refers to the **sophistication of the design information** that can be extracted from source code
- Ideally, the **abstraction level** should be **as high as possible**
- The **reverse engineering** process should be **capable of**
 - Deriving **procedural design representations** (a low-level abstraction)
 - **Program** and **data structure information** (a somewhat higher level of abstraction)
 - **Object models, data flow models** (a relatively high level of abstraction)
 - **Entity relationship models** (a high level of abstraction).



Reverse Engineering Cont.

- As the **abstraction level increases**, information will allow **easier understanding of the program**
- **Interactivity** refers to the **degree** to which the **human** is **"integrated"** with **automated tools** to **create an effective reverse engineering process**
- In most cases, as **the abstraction level increases**, **interactivity must increase**
- The **directionality** of the reverse engineering process is **one-way**, all **information extracted** from the source code is **provided to** the **software engineer**



Forward engineering

- **Forward engineering** is a process of **obtaining desired software from the specifications**, which were brought by reverse engineering
- Forward engineering is **same as software engineering process** with only one difference **it is carried out always after reverse engineering**
- In most cases, **forward engineering** does **not simply create a modern equivalent** of an **older program**
- **Rather**, new **user** and **technology requirements** are **integrated** into the reengineering effort
- The **redeveloped** program **extends** the **capabilities** of the **older application**



Software Configuration Management

The **SCM (Software Configuration Management)** is a set of **activities** that have been developed **to manage change** throughout the **software life cycle**

Purpose:

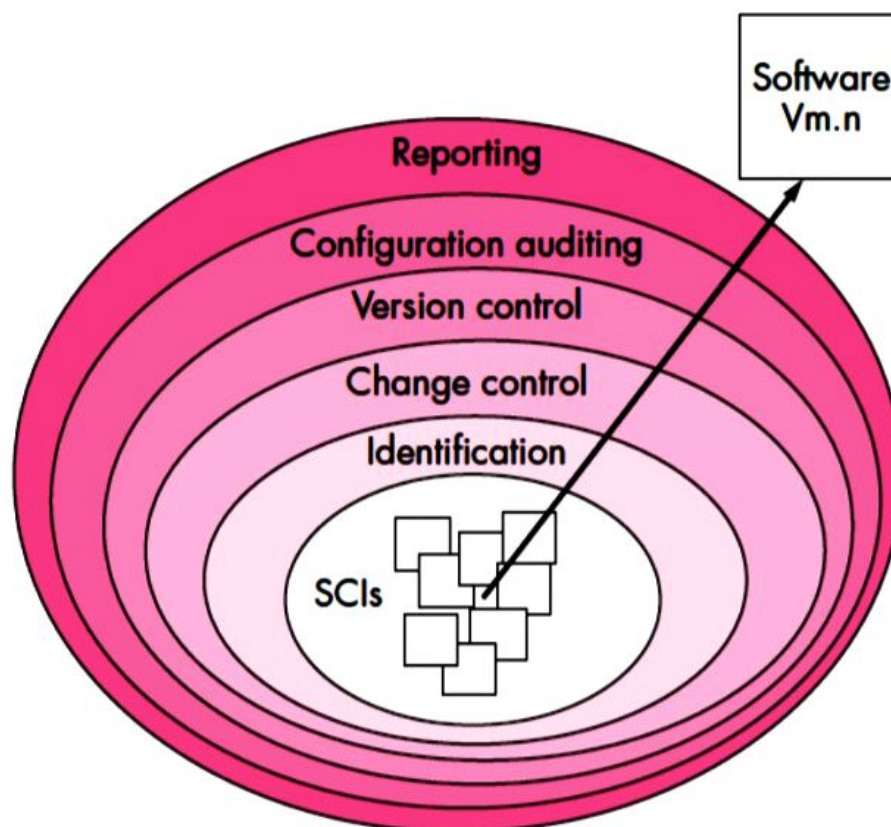
Systematically control changes to the configuration and **maintain** the **integrity** and **traceability** of the configuration throughout the system's life cycle

Four primary objectives

- To **identify** all **items** that collectively **define** the **software configuration**
- To **manage changes** to one or more of these items
- To facilitate the **construction** of different **versions** of an application
- To **ensure** that software **quality** is maintained as the **configuration evolves over time**



Layers of SCM Process



The SCM Process

- Referring to the figure, SCM tasks can be viewed as concentric layers
- **SCIs (Software Configuration Item)** flow outward through these layers throughout their useful life
- As an **SCI moves through a layer**, the **actions** implied by each SCM task **may or may not be applicable**
 - For example, when a new SCI is created, it must be identified.
 - However, if **no changes are requested** for the SCI, the **change control layer does not apply**
- The **SCI is assigned to a specific version** of the software (version control mechanisms come into play)
- A **record** of the **SCI** (its name, creation date, version, etc.) is **maintained** for configuration **auditing purposes**



What is Quality

Quality

Developed product meets it's specification

Quality Management

Ensuring that required level of product quality is achieved

Defining procedures and standards

Applying procedures and standards to the product and process

Checking that procedures are followed

Collecting and analyzing various quality data

Software Quality Assurance (SQA)

- ▶ **Software quality assurance** (also called **quality management**) is an **umbrella activity** that is applied throughout the software process
- ▶ It is planned and systematic pattern of **activities** necessary to **provide high degree of confidence in the quality**
- ▶ Software quality assurance (**SQA**) encompasses
 - ↳ An **SQA process**
 - ↳ Specific **quality assurance** and **quality control** tasks
 - ↳ **Effective software engineering** practice
 - ↳ **Control** of all software **work products**
 - ↳ A **procedure** to **ensure compliance** with software **development standards**
 - ↳ **Measurement** and **reporting mechanisms**

SQA Activities

- ▶ **Prepare** an **SQA plan** for a project
- ▶ **Participate** in the **development** of the project's **software process** description
- ▶ **Review** software engineering **activities** to **verify compliance** with the **defined software process**
- ▶ **Audit** designated **software work products** to verify compliance with those defined as part of the software process
- ▶ **Ensure** that **deviations** in software **work** and **work products** are **documented** and **handled** according to a documented procedure
- ▶ **Records** any **noncompliance** and reporting to **senior management**

SQA Techniques

- ▶ Statistical **quality assurance** implies **the following steps**:
 1. **Information** about software **defects** is **collected** and **categorized**
 2. **An attempt** is made to **trace each defect** to its underlying **cause**
 - Ex., non-conformance to specifications, design error, violation of standards, poor communication with the customer
 3. **Using** the **Pareto principle** (80 percent of the defects can be traced to 20 percent of all possible causes), **isolate the 20 percent** (the "vital few").
 4. Once the vital few causes have been identified, **move to correct the problems** that have caused the defects.
- ▶ Some of the defects are uncovered as software is being developed.
- ▶ Other are encountered after the software has been released.

SQA Techniques (Cont.)

Uncovered **errors can be tracked to** (one or more) of the **following causes**

- ▶ Incomplete or erroneous specifications (IES)
- ▶ Misinterpretation of customer communication (MCC)
- ▶ Intentional deviation from specifications (IDS)
- ▶ Violation of programming standards (VPS)
- ▶ Error in data representation (EDR)



Differentiation of SQA & SQC



Criteria	SQA	SQC
Definition	SQA is a set of activities for ensuring quality in software engineering processes (that ultimately result in quality in software products). The activities establish and evaluate the processes that produce products	SQC is a set of activities for ensuring quality in software products . The activities focus on identifying defects in the actual products produced
Focus	Process focused	Product focused
Orientation	Prevention oriented	Detection oriented
Breadth	Organization wide	Product/project specific
Scope	Relates to all products that will ever be created by a process	Relates to specific product
Activities	Process Definition and Implementation, Audits, Training	Reviews, Testing

Software Reviews (Formal Technical Reviews)

- ▶ A formal technical **review (FTR)** is a software **quality control activity** performed by software engineers (and others)
- ▶ **The objectives of an FTR are:**
 1. To **uncover errors** in **function, logic, or implementation**; for any representation of the software
 2. To **verify** that the **software under review meets** its **requirements**
 3. To **ensure** that the software has been **represented according to predefined standards**
 4. To **achieve software** that is **developed** in a **uniform manner**
 5. To **make projects** more **manageable**
- ▶ During the **FTR**,
 - ↳ a **reviewer** (the recorder) actively **records** all **issues** that have been raised
- ▶ These are **summarized at the end of** the review **meeting**, and a reviewed issues list is produced
- ▶ In addition, a formal technical **review summary report** is **completed**